

Module 4: Spring Boot Fundamentals, DevTools, Auditing, RestClient, Logging, Actuator, and OpenAPI

Questions

Q1. What is Spring Boot, and why is it popular for building Java backend applications?

Answer: Spring Boot is an opinionated framework built on top of the Spring Framework that simplifies application setup and deployment. It provides auto-configuration, starter dependencies, an embedded server, and production-ready features like Actuator out of the box. Instead of writing a lot of XML or Java configuration, a developer can add `spring-boot-starter-web` and start a runnable web application with a `main` method. It is popular because it reduces boilerplate, speeds up development, and makes microservices and cloud-native deployments easier.

Q2. What does `@SpringBootApplication` do internally?

Answer: `@SpringBootApplication` is a convenience annotation that combines `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`. `@Configuration` marks the class as a source of bean definitions. `@ComponentScan` tells Spring where to look for components, services, repositories, and controllers. `@EnableAutoConfiguration` triggers Spring Boot's auto-configuration mechanism, which inspects the classpath and existing beans to configure common infrastructure automatically, such as a datasource, embedded Tomcat, or Spring MVC. You can exclude specific auto-configurations using the `exclude` attribute.

Q3. What are Spring Boot starters, and how do they help?

Answer: Starters are pre-packaged dependency descriptors that bundle a set of compatible libraries for a specific use case. For example, `spring-boot-starter-web` includes Spring MVC, Tomcat, and Jackson, while `spring-boot-starter-data-jpa` includes Hibernate, Spring Data JPA, and a transaction manager. They reduce the need to hunt for compatible library versions and keep the build file clean. The `spring-boot-starter-parent` POM provides dependency management, default plugin configurations, and version alignment.

Q4. How do you create a simple REST controller in Spring Boot?

Answer: You annotate a class with `@RestController`, which combines `@Controller` and `@ResponseBody`. You then map HTTP methods using annotations like `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@DeleteMapping`. For example, `@GetMapping("/users/{id}") public User getUser(@PathVariable Long id)` handles a GET request and returns the user object, which Jackson serializes to JSON.

automatically. Using `@RestController` avoids the need to annotate every method with `@ResponseBody`.

Q5. What is the difference between `@Controller` and `@RestController`?

Answer: `@Controller` is the traditional Spring MVC annotation used when you want to return a view name that a `ViewResolver` renders. `@RestController` is a shortcut for `@Controller` plus `@ResponseBody`, meaning every method returns a domain object or data that is serialized directly to JSON, XML, or another response body. If you use `@Controller` for a REST API and forget `@ResponseBody`, Spring will look for a view with that name and likely throw a 404.

Q6. How does Spring Boot handle the embedded server, and how do you change the default port?

Answer: Spring Boot includes an embedded server, usually Tomcat, by adding `spring-boot-starter-web`. The application can run as a standalone JAR because the embedded server starts when the `SpringApplication.run` method is executed. You can change the port with `server.port=9090` in `application.properties`, or disable the web server entirely with `spring.main.web-application-type=none`. To use Jetty or Undertow, exclude Tomcat from the starter and add `spring-boot-starter-jetty` or `spring-boot-starter-undertow`.

Q7. What is auto-configuration, and how can you override or disable it?

Answer: Auto-configuration is Spring Boot's ability to guess the configuration a project needs based on the classpath and existing beans. It is implemented by classes annotated with `@ConditionalOnClass`, `@ConditionalOnMissingBean`, and similar conditions. You can override auto-configuration by defining your own beans with the same type, which takes precedence due to `@ConditionalOnMissingBean`. You can disable specific auto-configuration classes using `spring.autoconfigure.exclude` in `application.properties` or the `exclude` attribute of `@SpringBootApplication`.

Q8. How do you set up a Spring Boot project with Spring Boot Web MVC and Spring Data JPA?

Answer: Add the starters `spring-boot-starter-web` for the web layer and `spring-boot-starter-data-jpa` for persistence. Include a database driver such as `mysql-connector-j` or `postgresql`. Configure the datasource in `application.properties` with `spring.datasource.url`, `username`, `password`, and `driver-class-name`.

Define entity classes with `@Entity` and repository interfaces extending `JpaRepository`. Spring Boot auto-configures the datasource, transaction manager, and Hibernate. Use `@Transactional` in the service layer and keep controllers thin.

Q9. What is Spring Boot DevTools, and why should it not be used in production?

Answer: Spring Boot DevTools is a development-time dependency that provides automatic restart when code changes, LiveReload for browser refresh, and enhanced debugging features. It should not be used in production because it adds runtime overhead, can expose internal application details, and is designed only for the local development loop. In production builds, it should be excluded or marked with a non-transitive scope so that it does not end up in the deployed artifact.

Q10. How do you enable basic auditing in Spring Boot?

Answer: Enable JPA auditing with `@EnableJpaAuditing` on a configuration class and annotate entity fields with `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, and `@LastModifiedBy`. Spring Data JPA automatically fills these fields when entities are persisted or updated. For `@CreatedBy` and `@LastModifiedBy`, you provide an `AuditorAware` bean that returns the current user. This removes manual audit field population and helps with compliance and traceability.

Q11. What is Hibernate Envers, and when would you use it?

Answer: Hibernate Envers is an auditing module that tracks every insert, update, and delete on entities by maintaining separate revision tables. Annotating an entity with `@Audited` makes Envers store a snapshot of each change along with a revision number and timestamp. It is useful when you need a complete history of who changed what and when, such as in regulated domains or for debugging data issues. The trade-off is extra storage, write overhead, and the need to plan retention and archival of revision data.

Q12. How do you call a third-party REST API from a Spring Boot application?

Answer: You can use `RestTemplate` for synchronous calls, `WebClient` for reactive or non-blocking calls, or Feign for declarative clients. With `RestTemplate`, you inject a bean and call methods like `getForObject` or `exchange`. With `WebClient`, you build a request chain and subscribe to the response. With Feign, you declare an interface with method mappings and Spring Cloud generates the implementation. For new Spring Boot 3 projects, the `RestClient` API offers a modern, fluent synchronous alternative to `RestTemplate`.

Q13. What is the difference between `RestTemplate`, `WebClient`, `Feign`, and `RestClient`?

Answer: `RestTemplate` is synchronous and simple but is now in maintenance mode. `WebClient` is reactive and non-blocking, part of Spring WebFlux, and better for high-concurrency or streaming scenarios. `Feign` is a declarative Spring Cloud client that lets you define HTTP clients as interfaces, making service-to-service calls concise. `RestClient`, introduced in Spring Framework 6.1 and Spring Boot 3.2, is a modern synchronous client with a fluent API. Blocking a reactive pipeline with `RestTemplate` is a common mistake that kills throughput in WebFlux applications.

Q14. How do you handle failures, retries, and timeouts when calling external APIs?

Answer: External API calls should always have connection and read timeouts configured so that a slow downstream does not hang your service. Use `Resilience4j` for circuit breaking, retries with exponential backoff, rate limiting, and bulkheads. Wrap the client so that failures degrade gracefully, for example by returning cached or default data. Log correlation IDs to trace calls across services, and use `MockWebServer` or `WireMock` in tests instead of hitting real third-party endpoints.

Q15. What is the default logging framework in Spring Boot, and how do you configure it?

Answer: The default logging framework is `Logback`, accessed through the `SLF4J` facade. You can configure logging levels and output in `application.properties` with properties like `logging.level.root=INFO` and `logging.level.com.example=DEBUG`. For file logging, use `logging.file.name` or `logging.file.path`. For more control, provide a `logback.xml` or `log4j2.xml` file in the classpath. A common mistake is enabling `DEBUG` in production, which degrades performance and can leak sensitive information.

Q16. What is `SLF4J`, and why is it recommended over direct `Log4j` usage?

Answer: `SLF4J` is a logging facade that lets application code use a standard API while the actual logging implementation, such as `Logback` or `Log4j2`, is configured at deployment time. This means you can switch implementations by changing dependencies and configuration, without touching application code. It also supports parameterized messages such as `logger.info("Processing order {}", orderId)`, which avoid the cost of string concatenation when the log level is disabled.

Q17. What is Spring Boot Actuator, and what are its most useful endpoints?

Answer: Actuator is a production-ready feature that exposes operational endpoints for monitoring and managing the application. After adding `spring-boot-starter-actuator`, endpoints such as `/health`, `/info`, `/metrics`, `/env`, `/loggers`, `/beans`, and `/heapdump` become available. `/health` shows application health, `/metrics` exposes JVM and application metrics, and `/loggers` lets you change log levels at runtime. Exposing all endpoints publicly is dangerous, so only the endpoints needed for monitoring should be enabled and secured.

Q18. How do you secure Spring Boot Actuator endpoints?

Answer: Limit exposed endpoints using `management.endpoints.web.exposure.include` to only those required, such as `health`, `info`, and `metrics`. Use Spring Security to protect them with authentication and a dedicated role like `ACTUATOR_ADMIN`. Run Actuator over HTTPS and place it on a separate management port if needed. Endpoints like `/env`, `/beans`, `/heapdump`, and `/configprops` can reveal secrets or internal details, so they should be disabled or heavily restricted in production.

Q19. How do you create a custom health indicator in Spring Boot?

Answer: Implement the `HealthIndicator` interface and override the `health` method to check a specific dependency, such as a downstream API or a message broker. Register the class as a Spring bean. Spring Boot automatically includes it in the `/health` response. This is useful for readiness and liveness probes in Kubernetes, where the platform can restart or stop routing traffic to an unhealthy instance.

Q20. What is OpenAPI, and how does Swagger help document REST APIs?

Answer: OpenAPI is a standardized specification for describing REST APIs. Swagger provides tooling to design, document, and test APIs based on the OpenAPI specification. In a Spring Boot project, you can use SpringDoc OpenAPI to generate documentation automatically from controllers, DTOs, and annotations. It produces an interactive UI where developers can see endpoints, parameters, request bodies, and response schemas, and even try calls directly.

Q21. Should Swagger UI be enabled in production, and why or why not?

Answer: Swagger UI should generally be disabled or restricted in production. While it is helpful during development and integration testing, it exposes the complete API contract, including endpoints, parameter names, and response structures, which can aid attackers. The recommended approach is to enable Swagger only in non-production profiles or protect it behind authentication and authorization.

Q22. What are Spring profiles, and how do you use them?

Answer: Spring profiles let you define environment-specific beans and configuration. You activate a profile with `spring.profiles.active=prod` in `application.properties` or via command-line arguments. Beans annotated with `@Profile("dev")` only load when that profile is active. You can also have separate property files like `application-dev.properties` and `application-prod.properties`. Profiles help keep database URLs, log levels, feature flags, and secrets isolated per environment.

Q23. How do you externalize configuration and secrets in a Spring Boot application?

Answer: Configuration should be moved out of the application code into property files, environment variables, or external config servers. Spring Boot supports `application.properties`, `application.yml`, environment variables, and command-line arguments with a defined precedence order. Secrets such as database passwords and API keys should not be stored in source control. Use Spring Cloud Config, AWS Secrets Manager, Azure Key Vault, or HashiCorp Vault to inject secrets securely at runtime.

Q24. What is the Spring MVC request lifecycle?

Answer: An incoming request first reaches the `DispatcherServlet`, which consults handler mappings to find the right controller method. The method processes the request, possibly using `@RequestBody` to deserialize JSON into objects and `@PathVariable` or `@RequestParam` to extract values. The result is returned, and if the controller uses `@RestController`, it is written directly to the response body. Interceptors can run before and after the controller, and `@ControllerAdvice` can handle exceptions globally.

Q25. What is `ResponseEntity`, and when should you use it?

Answer: `ResponseEntity` is a Spring class that represents the full HTTP response, including status code, headers, and body. It gives complete control over what is returned to the client. For example, `ResponseEntity.created(location).body(user)` returns HTTP 201 with a `Location` header, while `ResponseEntity.status(HttpStatus.NOT_FOUND).build()` returns a 404. It is preferred over returning raw domain objects when you need to set custom headers or status codes.

Q26. How do you handle exceptions globally in Spring Boot?

Answer: Use `@RestControllerAdvice` or `@ControllerAdvice` with methods annotated by `@ExceptionHandler`. A global advice class catches exceptions thrown by controllers and

returns consistent error responses. For example, a `ResourceNotFoundException` can be mapped to HTTP 404, while a `ValidationException` can be mapped to HTTP 400. This keeps controllers clean and ensures the API always returns a predictable error format.

Q27. What is `application.properties` vs `application.yml`, and which should you use?

Answer: Both files store externalized configuration. `application.properties` uses simple key-value pairs, while `application.yml` uses nested YAML syntax, which is cleaner for hierarchical configuration. The choice is mostly a team preference. If the configuration is deeply nested, such as multiple datasources or logging appenders, YAML is usually more readable. Properties files are simpler for flat configurations.

Q28. What is the Spring Boot main class, and why does it need `SpringApplication.run()`?

Answer: The main class is the entry point annotated with `@SpringBootApplication`. Calling `SpringApplication.run(MyApplication.class, args)` bootstraps the Spring application context, starts the embedded web server, runs command-line runners, and initializes all beans. It is what turns a plain Java application into a running Spring Boot service. The class is usually placed at the root package so component scanning covers the entire project.

Q29. How do you write integration tests in Spring Boot?

Answer: Use `@SpringBootTest` to load the full application context, `@WebMvcTest` for controller-layer tests with mocked services, and `@DataJpaTest` for repository tests against an embedded database. Use `TestRestTemplate` or `WebTestClient` to call endpoints. For external dependencies, use `@MockBean` or tools like WireMock. Tests should be fast and deterministic, so prefer sliced tests over full context tests when possible.

Q30. What is the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?

Answer: All four are stereotype annotations that mark classes as Spring-managed beans. `@Component` is the generic one. `@Service` is used for business logic classes and helps document intent. `@Repository` is used for data access and enables Spring's exception translation for persistence exceptions. `@Controller` is used for MVC controllers. Functionally they all result in bean creation during component scanning, but the specialized annotations carry semantic meaning and enable framework features.

Q31. What is dependency injection, and why is constructor injection preferred?

Answer: Dependency injection is a design pattern where Spring supplies a bean's dependencies instead of the bean creating them itself. Constructor injection is preferred because it makes dependencies explicit and immutable, allows easy unit testing without reflection, and ensures the object is fully initialized before use. Field injection hides dependencies and makes testing harder, while setter injection can leave an object in a partially constructed state.

Q32. How do you schedule tasks in Spring Boot?

Answer: Add `@EnableScheduling` on a configuration class and use `@Scheduled` on methods to run them at fixed intervals, fixed delays, or using cron expressions. For example, `@Scheduled(cron = "0 0 * * * *")` runs every hour. For reliability and clustering, consider using an external scheduler or Quartz with a JDBC job store so that jobs are not duplicated across multiple application instances.

Q33. What is the difference between `@Async` and `@Scheduled`?

Answer: `@Async` marks a method to run in a separate thread from a task executor, returning immediately to the caller. It is useful for fire-and-forget operations like sending emails or notifications. `@Scheduled` triggers a method repeatedly based on a timer. A common mistake is calling an `@Async` method from within the same bean, which bypasses the Spring proxy and runs synchronously.

Q34. How do you version REST APIs in Spring Boot?

Answer: Common approaches include URI versioning (`/v1/users`, `/v2/users`), request header versioning (`Accept: application/vnd.company.v1+json`), and query parameter versioning (`?version=1`). URI versioning is the simplest and most visible. Header versioning keeps URLs clean but is harder to test in a browser. The choice should be documented and consistent across the API surface.

Q35. What is content negotiation in Spring Boot?

Answer: Content negotiation is the mechanism by which a client and server agree on the format of the response, usually JSON or XML. Spring Boot uses the `Accept` header to decide which `HttpMessageConverter` to use. Jackson is configured by default for JSON, and adding Jackson XML or JAXB dependencies enables XML responses. Producing the wrong format because of an `Accept` header mismatch is a subtle bug that appears during client integration.

Q36. How do you validate request input in Spring Boot?

Answer: Use Bean Validation by adding the `spring-boot-starter-validation` dependency and annotating DTO fields with constraints like `@NotBlank`, `@Email`, `@Min`, `@Max`, and `@Valid`. In the controller, annotate the request body parameter with `@Valid` so that validation runs before the method executes. Validation failures result in a `MethodArgumentNotValidException`, which can be handled globally to return a structured error response.

Q37. What is the purpose of `CommandLineRunner` and `ApplicationRunner`?

Answer: Both interfaces let you run logic once the Spring application context is fully initialized. `CommandLineRunner` accepts raw command-line arguments as a string array, while `ApplicationRunner` exposes them as structured `ApplicationArguments`. They are commonly used for one-time setup, data seeding, or checking external dependencies at startup. Multiple runners can be ordered with `@Order`.

Q38. How do you package and deploy a Spring Boot application?

Answer: Spring Boot applications are typically packaged as an executable JAR using the Spring Boot Maven or Gradle plugin. The JAR includes the embedded server, so you deploy it by running `java -jar app.jar`. For containers, you can build a Docker image with a Dockerfile or use `spring-boot:build-image` to create an optimized image. Configuration is externalized through environment variables or mounted config files so the same artifact can run in any environment.

Q39. What is graceful shutdown in Spring Boot?

Answer: Graceful shutdown allows the application to finish processing active requests before stopping when it receives a termination signal. In Spring Boot 2.3+, you can enable it with `server.shutdown=graceful`. The embedded server stops accepting new requests, waits for in-flight requests up to a configured timeout, and then shuts down. This prevents dropped requests during rolling deployments and Kubernetes pod terminations.

Q40. What is the Spring Boot fat JAR, and how is it different from a regular JAR?

Answer: A fat JAR, also called an uber JAR, is a Spring Boot executable JAR that includes the application classes and all runtime dependencies, including the embedded server. A regular JAR usually contains only the project classes and needs external dependencies on the classpath. The fat JAR is self-contained, making deployment simple because you only need to run `java -jar`.

Q41. How do you add a custom Actuator endpoint?

Answer: Create a class annotated with `@Endpoint` and a unique ID. Define operations using `@ReadOperation`, `@WriteOperation`, and `@DeleteOperation`. Register the class as a Spring bean. For example, a `@ReadOperation` method can return a custom metric or configuration value. Make sure the endpoint is exposed in `management.endpoints.web.exposure.include` and protected by security rules.

Q42. What is Micrometer, and how does it relate to Actuator?

Answer: Micrometer is a metrics facade that provides a vendor-neutral API for recording timers, counters, gauges, and distributions. Spring Boot Actuator exposes Micrometer metrics through the `/metrics` endpoint and can ship them to backends like Prometheus, Datadog, CloudWatch, or Graphite. This lets you monitor application performance, throughput, and errors using the tools your organization already has.

Q43. How do you change the Actuator base path and port?

Answer: You can change the Actuator web base path with `management.endpoints.web.base-path=/admin`, so endpoints appear under `/admin/health` instead of `/actuator/health`. You can also run Actuator on a separate management port with `management.server.port=8081`, which isolates operational endpoints from public application traffic and makes security rules easier to apply.

Q44. What is the difference between `@Value` and `@ConfigurationProperties`?

Answer: `@Value` injects a single property using SpEL expressions, such as `@Value("${server.port}")`. `@ConfigurationProperties` binds a group of related properties to a typed bean, which is better for structured configuration like a datasource or an external API client. For example, `app.mail.host`, `app.mail.port`, and `app.mail.username` can bind to a single `MailProperties` class. `@ConfigurationProperties` supports relaxed binding and JSR-303 validation.

Q45. How do you implement pagination in a Spring Boot REST API?

Answer: Add a `Pageable` parameter to the controller or repository method. Spring resolves `Pageable` from query parameters like `?page=0&size=10&sort=name,asc`. The repository returns a `Page<T>` that includes content, total elements, total pages, and metadata. Always cap the page size to prevent abuse and avoid returning huge responses that exhaust memory or slow serialization.

Q46. What is a DTO, and why should you return DTOs instead of entities from controllers?

Answer: A DTO is a plain object that carries only the data needed by the API consumer. Returning entities directly couples the API contract to the database schema, can trigger `LazyInitializationException`, may expose internal fields, and makes future schema changes risky. Mapping entities to DTOs inside the service layer gives full control over the exposed shape and avoids serialization problems.

Q47. How do you handle file uploads in Spring Boot?

Answer: Accept `MultipartFile` as a controller parameter, usually with `@RequestParam("file")` or `@ModelAttribute`. Use `transferTo` to save the file or stream it directly to cloud storage like S3. For large files, stream the content instead of loading it fully into memory, and configure `spring.servlet.multipart.max-file-size` and `max-request-size` to enforce limits. Store files in external storage rather than local disk in containerized environments.

Q48. What is an API Gateway, and how does it fit with Spring Boot microservices?

Answer: An API Gateway sits in front of microservices and handles cross-cutting concerns like routing, load balancing, authentication, rate limiting, SSL termination, and request/response transformation. Spring Cloud Gateway is a common choice in the Spring ecosystem. Each Spring Boot service exposes its own endpoints, and the gateway routes client requests to the right service, centralizing security and observability.

Q49. How do you implement distributed tracing in Spring Boot?

Answer: In Spring Boot 3, use Micrometer Tracing with Brave or OpenTelemetry to propagate trace and span IDs across service calls. Add the tracing starter and configure an exporter such as Zipkin or Jaeger. The trace ID is included in logs via MDC, which helps you follow a request across multiple services. In Spring Boot 2.x this was often done with Spring Cloud Sleuth.

Q50. What are the key things to check before deploying a Spring Boot application to production?

Answer: Before production deployment, disable DevTools and Swagger UI or restrict them, secure Actuator endpoints, externalize secrets, set appropriate log levels, enable health checks, configure graceful shutdown, set reasonable connection and thread pool sizes, run integration tests, verify database migrations, and ensure the artifact is built with the production profile. Also confirm that metrics and centralized logging are wired up.

Scenario based questions

S1. A Spring Boot application starts up locally but fails to start in production with a DataSource bean creation error. How do you investigate?

Answer: Start by reading the full stack trace to identify whether the failure is a missing URL, invalid credentials, unreachable host, driver not on the classpath, or a port/firewall issue. Compare the production `application.properties` or environment variables with the local configuration. Check that the correct profile is active and that secrets are being injected properly by Vault or the config server. Verify the database driver dependency is included in the production build. If the app runs in a container, confirm that the container can reach the database and that DNS resolution works. Finally, test connectivity with a simple JDBC connection from the production environment before restarting the application.

S2. A REST endpoint works in development but returns 404 in production. What could be wrong?

Answer: A 404 usually means the request URL does not match any registered handler. Check whether the production base path or servlet context path is different, such as `server.servlet.context-path=/api`. Verify the HTTP method is correct and that the controller is in a package scanned by Spring. If there are multiple `@RequestMapping` variants or a conflicting global path configuration, they may behave differently under another profile. Also confirm that no security filter is blocking the request and returning a misleading 404 instead of 403.

S3. An API that handles file uploads starts throwing OutOfMemoryError under load. How do you fix it?

Answer: Loading the entire multipart file into memory for every request consumes heap space quickly. The fix is to stream the file directly to disk or cloud storage instead of keeping it in memory. Configure `spring.servlet.multipart.file-size-threshold` so large files are written to disk, and set `max-file-size` and `max-request-size` limits to reject oversized uploads. Use `MultipartFile.getInputStream()` and upload chunks to S3 or similar storage. In containerized environments, avoid writing to ephemeral local disk; send the stream straight to object storage.

S4. A Spring Boot service calls an external payment API, and payments sometimes time out. What do you do?

Answer: First configure explicit connection and read timeouts on the HTTP client so the call fails fast instead of blocking threads indefinitely. Add retries with exponential backoff for transient failures, but avoid retrying idempotent mutations blindly without idempotency keys. Wrap the client with a circuit breaker such as Resilience4j so failures do not cascade. Log every outbound call with a trace ID and the response time. If the external API is unreliable, consider an asynchronous queue-

based integration where the payment status is polled or pushed later.

S5. A microservice using Feign starts failing with `NoSuchBeanDefinitionException`. How do you debug it?

Answer: `NoSuchBeanDefinitionException` means Spring could not create the Feign client bean. Verify that `@EnableFeignClients` is on the main class or a configuration class and that the base package is correct. Confirm the interface is annotated with `@FeignClient` and has a valid name and URL or service ID. Check that `spring-cloud-starter-openfeign` is on the classpath. If the service uses a custom Feign configuration, ensure it is not accidentally excluding the default beans needed for auto-configuration.

S6. A Spring Boot application running on Kubernetes is being killed before requests finish. How do you handle it?

Answer: Kubernetes sends a `SIGTERM` to the pod when scaling down or redeploying. If the application exits immediately, in-flight requests are dropped. Enable graceful shutdown with `server.shutdown=graceful` and set `spring.lifecycle.timeout-per-shutdown-phase` to a value larger than the longest expected request. Configure the pod's `terminationGracePeriodSeconds` to match. The container should keep running until active requests complete or the grace period expires, after which Kubernetes sends `SIGKILL`.

S7. A newly deployed feature works for some users but returns stale data for others. What could be happening?

Answer: Stale data for some users often points to caching issues. If each application instance has a local cache, one instance may update the database without invalidating the cache on other instances. The fix is to use a shared cache such as Redis or to publish cache invalidation events across the cluster. Also check whether the cache TTL is too long, whether the cache key includes all relevant parameters, and whether the feature depends on a read replica that lags behind the primary database.

S8. A Spring Boot batch job processes orders, but the same order is processed twice when the job runs on two pods. How do you prevent duplicate processing?

Answer: Duplicate processing happens because each pod independently polls the same work queue or table. The solution depends on the architecture. If using a scheduler, make it a singleton with a distributed lock via `ShedLock` or Quartz JDBC job store. If pulling from a message broker, use a consumer group so each message is processed by only one consumer. If reading from a database, mark rows as `PROCESSING` with an atomic update, or use a `processed_at` timestamp and a unique constraint on the business key to ensure idempotency.

S9. Actuator /health returns UP even though the database is unreachable. How do you fix it?

Answer: By default, Spring Boot health checks only verify the datasource if it is auto-configured and the application uses it. If the health indicator is disabled or the datasource is custom, the database may not be checked. Add a custom `HealthIndicator` that performs a lightweight query like `SELECT 1` against the database, or enable detailed health with `management.endpoint.health.show-details=always` and include `db` in the exposed endpoints. For Kubernetes liveness and readiness probes, ensure the readiness probe points to `/health/readiness` and the liveness probe to `/health/liveness` so traffic is routed away before the pod is restarted.

S10. A Spring Boot application shows high CPU after a deployment. How do you troubleshoot it?

Answer: High CPU after a deployment usually means inefficient loops, excessive garbage collection, or a hot code path. Start by collecting thread dumps and heap dumps, and look at JVM metrics through Actuator or a profiler. Check whether a new query runs full table scans, whether a cache was disabled, or whether a new loop processes an unexpectedly large list. Review recent changes for missing pagination, inefficient algorithms, or blocking calls on event loops. Revert the deployment if the root cause cannot be found quickly and business impact is high.

S11. A developer stores database passwords in `application.properties`. What do you recommend?

Answer: Storing passwords in source-controlled properties is a security risk. Move secrets to an external secret store such as HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, or Spring Cloud Config with encryption. At runtime, the application reads the secret through environment variables or the config server. If you must use property files locally, keep them out of version control with a `.gitignore` entry and use a secrets manager for all non-local environments. Rotate credentials regularly and audit access.

S12. A service using `@Async` runs synchronously when called from within the same bean. Why?

Answer: Spring AOP creates a proxy around the bean to handle `@Async`, `@Transactional`, and similar annotations. When the method is called from within the same bean, the call bypasses the proxy and runs directly, so the async behavior is ignored. The fix is to move the async method to another bean and inject it, or use `AopContext.currentProxy()` to invoke the method through the proxy. Constructor injection and clear separation of concerns reduce this problem.

S13. A Spring Boot application logs are not reaching the centralized logging system. What do you check?

Answer: Verify that the application writes logs to stdout or to a file that the log shipper can read. Check the logging configuration for incorrect appenders or file paths that do not exist in the container. Confirm the log level is not set too high, filtering out the messages you expect. In Kubernetes, make sure the container runtime captures stdout and that the log aggregation agent such as Fluentd or Fluent Bit is running. For structured JSON logging, ensure the format matches what the central system expects and that the MDC fields such as trace ID are included.

S14. An OpenAPI-generated contract does not match the actual API response. How do you fix it?

Answer: The contract becomes stale when DTOs, controller signatures, or annotations change without updating the OpenAPI generation. Use SpringDoc or Springfox annotations such as `@Operation`, `@ApiResponse`, `@Schema`, and `@Parameter` to document behavior explicitly. Generate the OpenAPI spec as part of the build and fail the build if it drifts from a committed baseline. In CI, run contract tests that verify the generated spec against real controller responses. Restrict manual edits of the spec so that the code remains the single source of truth.

S15. A Spring Boot application needs to support two databases. How do you configure it?

Answer: Define two separate `DataSource` beans, each with its own properties prefix using `@ConfigurationProperties`. Create two `EntityManagerFactory` beans, each scanning a different package of entities, and two `JpaTransactionManager` beans. Mark one set as `@Primary` so that default autowiring works for the main database. Use `@Qualifier` when injecting the secondary datasource, entity manager, or transaction manager. Avoid mixing entities from both databases in the same transaction unless you use a distributed transaction manager such as Atomikos or accept eventual consistency with application-level coordination.

S16. A production endpoint exposes sensitive user data in stack traces. How do you prevent it?

Answer: Configure a global exception handler with `@RestControllerAdvice` that catches exceptions and returns a sanitized error response containing only a generic message and a correlation ID. Never return raw stack traces, SQL errors, or internal class names to API consumers. Set `server.error.include-stacktrace=never` and `server.error.include-message=never` in production. Log the full exception with the correlation ID internally so developers can debug without leaking details.

S17. A Spring Boot application deployed as a Docker container is slow to start. How do you optimize it?

Answer: Slow startup is often caused by classpath scanning, large dependency trees, or inefficient database migration checks. Reduce startup time by moving to Java 17 or later, using layered JARs or CDS archives, trimming unnecessary dependencies, and lazy-initializing beans where appropriate. Pre-run database migrations during the build or as an init container instead of blocking startup. Build optimized container images with `spring-boot:build-image` or a custom Dockerfile based on a small JRE. Avoid running in DEBUG logging in production.

S18. A Spring Boot API returns different responses in dev and prod for the same request. How do you investigate?

Answer: Compare the active profiles and the resolved property values using `/actuator/env` if available. Check for environment-specific beans enabled by `@Profile` and for conditional properties that differ between environments. Verify the database state, including schema version, seed data, and replication lag if a read replica is used. Look for feature flags or cached data that may differ. Capture the request and response in both environments and diff them to isolate whether the difference is in data, serialization, or business logic.

S19. A Spring Boot service under heavy load starts returning connection timeout errors. What do you check?

Answer: Check the embedded server thread pool configuration, the database connection pool size, and the HTTP client connection pool for outbound calls. If any pool is too small, requests queue up and eventually time out. Use metrics to see whether the database is the bottleneck or whether an external API is slow. Tune `server.tomcat.threads.max`, HikariCP `maximum-pool-size`, and client timeouts. Add circuit breakers and bulkheads to prevent failures from cascading. If the load is legitimate, scale horizontally behind a load balancer.

S20. A new developer enables all Actuator endpoints for easier debugging. What do you tell them?

Answer: Exposing all endpoints, especially `/env`, `/beans`, `/heapdump`, `/configprops`, and `/loggers`, leaks configuration values, internal structure, and potentially secrets. The right approach is to expose only the endpoints needed for monitoring, such as `/health`, `/info`, `/metrics`, and `/prometheus`, and protect them with Spring Security and a dedicated management role. Use a separate management port or restrict network access so operational endpoints are not reachable from the public internet.

S21. A Spring Boot application receives a request with a JSON body, but the fields are null even though the JSON has values. Why?

Answer: This usually happens when the JSON property names do not match the Java field names and there are no matching getters and setters, or when the DTO lacks a default constructor. If you use Lombok, verify that `@Getter`, `@Setter`, and `@NoArgsConstructor` are present and that the annotation processor is configured in the build. Another cause is a custom `HttpMessageConverter` or `ObjectMapper` configuration that disables default deserialization. Use `@JsonProperty` if the API names differ from Java naming conventions.

S22. A scheduled job in a Spring Boot application works on one instance but runs twice in a clustered deployment. How do you fix it?

Answer: By default, Spring's scheduler runs on every node independently. In a cluster you need a distributed lock so only one instance executes the job. Options include `ShedLock`, which uses a database or Redis to coordinate, or Quartz with a JDBC job store. If the job is triggered by a message broker, use a consumer group so each message is processed exactly once. Avoid relying on cron expressions alone in clustered environments because every node will fire at the same time.

S23. A Spring Boot service is supposed to send an email after user registration, but emails are not sent when the registration API is under load. How do you debug?

Answer: If the email sending is synchronous, the external SMTP server can become a bottleneck, causing requests to time out or fail. Move the email logic to an `@Async` method or a message queue so registration completes quickly. Verify the thread pool used by `@Async` is large enough and has a queue capacity. Check logs for SMTP exceptions and configure a circuit breaker around the email provider. For reliability, persist the email intent in an outbox table or queue and process it with a retry mechanism.

S24. A Spring Boot application uses Swagger, and the security team asks to remove it from production. What is the best approach?

Answer: The best approach is to disable Swagger UI in production by using a Spring profile, such as adding the `SpringDoc` configuration only when the `dev` or `qa` profile is active. Alternatively, you can restrict the Swagger endpoints with Spring Security so only authorized internal users can access them. Removing the dependency entirely from the production build is the safest option if it is not needed there. Keep the OpenAPI spec generation in CI so documentation stays accurate, but do not expose the interactive UI publicly.

S25. A Spring Boot service needs to call another internal service, but the target service has frequent short outages. How do you make the integration resilient?

Answer: Add a circuit breaker like Resilience4j around the client so failures fail fast instead of blocking threads. Combine it with retries that have exponential backoff and jitter, but only for safe, idempotent operations. Use a fallback strategy that returns cached data, a default response, or a queued intent when the downstream service is unavailable. Set sensible connection and read timeouts on the HTTP client. Monitor the circuit breaker state with Actuator and alert when it opens frequently, because that signals a deeper downstream problem.